Breakpoint — Hyphenate Track Build Plan

Oct 23, 2026 · @Name

Overview

Breakpoint is an autonomous reconciliation team for a retail brokerage: it finds trade and cash breaks across five sources, resolves the ones it has learned, and asks a human only when it meets something new.

- **Seats it fills:** trade reconciliation analyst and settlements/cash ops analyst at a retail brokerage.
- **Company (mock):** Meridian Securities, a fictional broker-dealer. 200 client accounts, 40 US equities, about 80 business days (Jul 1 to Oct 23, 2026). Q3 is closed; October is open.
- **Core idea:** matching is deterministic code. Agents handle everything after a break is found: root cause, which source to trust, who to contact, what to correct, and getting sign-off. New fixes are learned from humans as reviewable playbooks.
- **Accuracy rule:** the LLM never produces a number that gets posted. Every amount comes from deterministic calculators; the LLM chooses, explains and drafts.

Demo story (2 minutes)

1. Replay August in fast-forward. Known breaks resolve themselves, each citing the playbook it used and who taught it.
2. A 2:1 stock split hits. No playbook fits, so the agent stops, asks the ops lead, and turns her answer into a draft playbook.
3. The controller approves it. A second split in October is fixed automatically, citing the August lesson.
4. A judge injects a break of their choice from the UI and watches it get handled.

Mock world and input streams

Eight inputs make up the company: five systems that can disagree, plus the reference data, contract and people that settle the disagreements. The generator builds one true world first, derives each system's view of it, then plants breaks.

#	Source	What it says	Key fields	Cadence
1	Client orders (OMS)	What the client asked for	order_id, account_id, symbol, side, qty, order_type, limit_px, submitted_at, status	Real time
2	Executions (executing broker, FIX-style)	What actually traded	exec_id, order_id, fill_qty, fill_px, venue, exec_time, exec_type (new, cancel, correct)	Real time
3	Internal ledger (books and records)	What Meridian recorded	txn_id, date, postings (account, asset, amount), metadata (source_ref, reason, actor)	On post
4	Custodian / clearing statement	What settled and is held	positions (account, symbol, qty), cash balances, settlements (trade_ref, settle_date, net_amount)	Daily file at end of day
5	Bank (firm account + client ACH)	Money in and out	transfer_id, account_id, direction, amount, status, return_code	Event
6	Reference and market data	Prices, corporate actions, symbols	symbol, cik, close_px; action type (split, dividend), ratio, ex_date, pay_date	Daily
7	Fee schedule (client agreement)	The contract	tier, per-trade fee, per-share rate, min, max, effective_from	Versioned; one tier change mid-quarter
8	People directory + inbox	Who owns what, who approves what	role, name, owns, approval_limit, reply SLA	Static; replies simulated

People (LLM-played, answering from the true world)

Role	Persona	Owns
Ops lead	Dana Ortiz	Teaches new fixes; first contact for unknown breaks
Controller	Sam Lee	Approves entries over $500, all client money movements, all new playbooks
Executing broker desk	Northgate Execution	Confirms fills, busts, corrections
Custodian ops	Harbor Clearing	Confirms positions, settlements, corporate actions
Client service	Alex Kim	Sends client notices
Clients	Simulated account holders	Respond to cash calls

Mock data options and stack

Use real software from Hyphenate's list where it adds credibility (ledger, bank, symbols), and our own stateful mocks where we need to control and inject data.

Need	Pick	Fallback	Why
Ledger (system of record)	Formance Ledger (Docker)	Postgres double-entry tables with a balance check	Enforces double entry, stores metadata for the audit trail, supports idempotency keys; on Hyphenate's list
Bank and client ACH	Increase sandbox	In-app mock bank service	Real ACH lifecycle, including returns (R01) and corrective transfers
OMS, executions, custodian	Our own FastAPI mock services, one per source, each with its own API key	Mockoon or Prism from an OpenAPI spec	We need state and live injection; Mockoon and Prism serve static examples

Need	Pick	Fallback	Why
Symbols	SEC EDGAR company tickers (real CIKs)	Hard-coded list of 40	Makes the world real enough
Prices	Real daily closes, fetched once and cached	Seeded random walk	Either way, recorded as real or simulated
Second domain (generality)	BenchRec cash reconciliation dataset	Stripe sandbox payouts vs bank	Proves a new domain loads by config only
App state	Postgres: breaks, cases, proposals, playbooks, append-only events	—	One place for the dashboard to read
Backend	Python 3.12, FastAPI, Pydantic, SQLAlchemy, Decimal everywhere	—	Fast to build; exact money math
Agents	Google ADK or Claude tool use, behind one thin llm.py wrapper	—	Pick in Step 0; the wrapper keeps it swappable
Frontend	React (Vite), Tailwind, shadcn/ui, Recharts; polls the API every 2 s	Streamlit	Polished enough for the video
Run	docker-compose + Makefile: make world SEED=42, make replay, make test, make demo	—	One command per demo step

Break taxonomy and answer key

Four known break types have playbooks from induction, two harmless differences must clear on their own, and two novel types force the agent to ask a human.

ID	Break	Sources that disagree	Who is right	Correct fix	Demo role	Planted
B1	Partial fill booked as full	Ledger 500 shares vs executions 320	Executions	Reverse 180 shares and the cash; notify client service	Known	6
B2	Commission charged at the wrong tier	Ledger fee vs fee schedule	Fee schedule (effective date matters)	Adjusting entry refunding the difference; client notice	Known	6
B3	Duplicate execution report	Ledger booked twice; custodian settled once	Custodian	Reverse the duplicate posting; broker desk confirms	Known	4
B4	Dividend not credited	Custodian paid cash; ledger has nothing	Custodian + corporate action feed	Post dividend = rate x shares held on record date	Known	3
H1	Settlement timing (T+1)	Trade on T, custodian settles T+1	Both	Auto-clear when the settle date passes	Harmless	Natural
H2	Price rounding	Difference of $0.01 or less per trade	Both	Auto-clear with reason	Harmless	10
N1	2:1 stock split not applied in ledger	Custodian qty doubled; ledger unchanged	Custodian + corporate action feed	Double the position, halve cost per share, no cash impact; apply to every holder	Novel: the main moment (Aug), then auto (Oct)	2
N2	ACH deposit returned (R01) after shares were bought	Bank shows return; ledger shows cash	Bank	Reverse the deposit, book a client receivable, request funds; controller decides on liquidation if unpaid after 3 days	Novel: outreach + money	2

ID	Break	Sources that disagree	Who is right	Correct fix	Demo role	Planted
S1	Trade booked to the wrong account	Equal and opposite breaks in two accounts	Executions + order	Move the trade between accounts	Stretch	2
S2	Broker correction not reflected in ledger	Executions show correct; ledger shows original	Executions	Reverse and rebook	Stretch	2

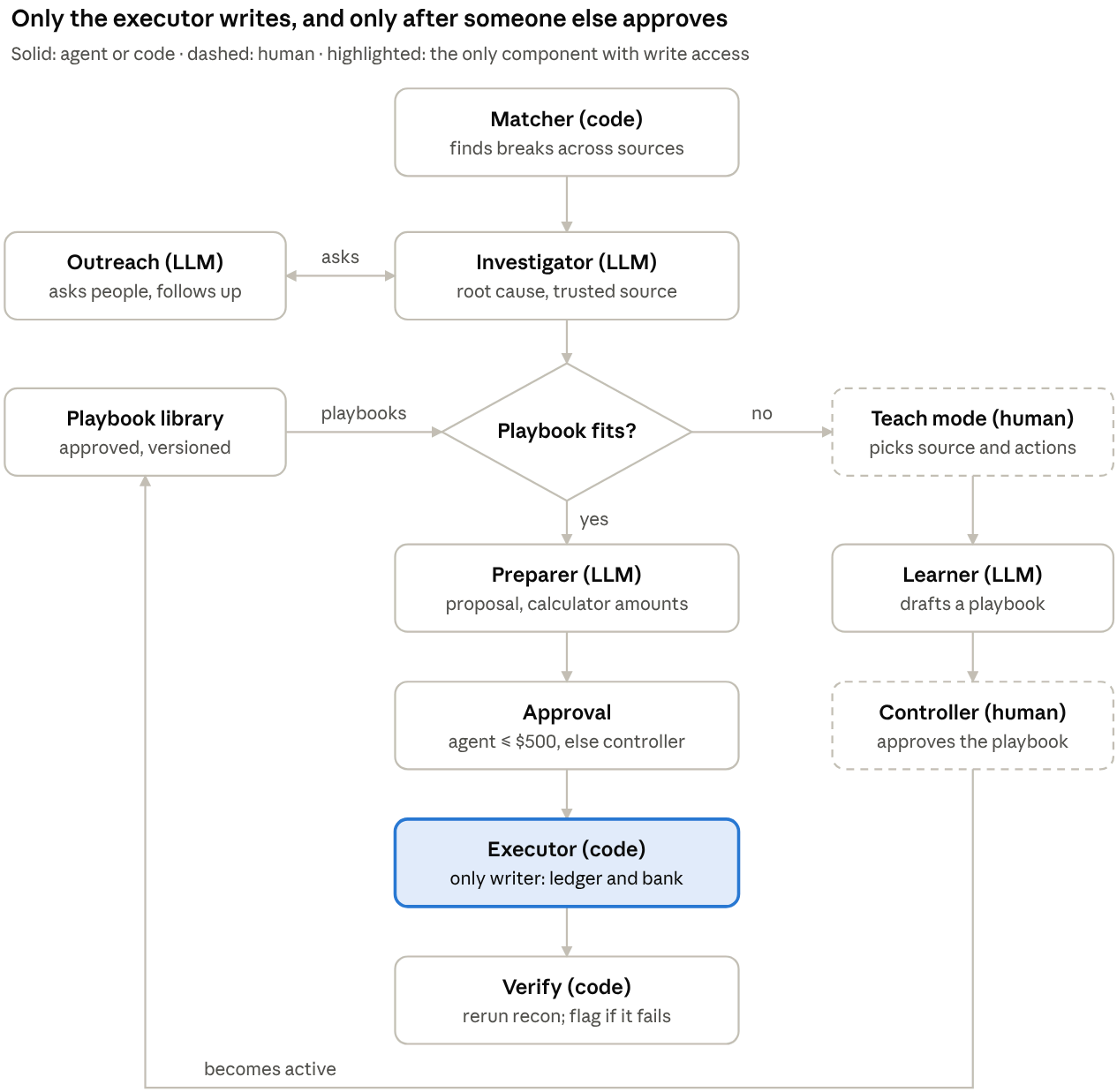
Answer key (one row per planted break): break_id, type, injected_on, affected records (source + ids), expected detection keys, root-cause source, expected fix (action types + exact amounts), expected final status (completed, auto-cleared, needs human).

Agents and segregation of duties

No agent can both propose and approve a change, and only one component, the executor, holds write access to the ledger and bank.

Agent	Type	Can read	Can do	Cannot
Matcher	Deterministic code	All sources (read-only keys)	Create breaks	Anything else
Investigator	LLM	Sources, breaks, playbooks, event log	Write case notes, root cause, playbook match	Propose fixes, message people, post
Preparer	LLM + deterministic calculators	Case evidence, playbooks	Create proposals: journal entry, source correction request, transfer	Approve or execute
Outreach	LLM	Directory, case summary	Send messages, follow up, record replies	Propose, post, move money

Agent	Type	Can read	Can do	Cannot
Approver	LLM bound by a coded policy	Proposal, evidence, playbook	Approve proposals of $500 or less from active playbooks	Anything over $500, client money movement, new playbooks
Controller	Human, in the UI	Everything	Approve anything; approve or retire playbooks	—
Executor	Deterministic code	Approved proposals only	Post to ledger, send bank transfers, apply source corrections	Execute anything unapproved or changed after approval
Learner	LLM	Teach sessions, historical cases, SOP	Draft playbooks	Activate playbooks
Auditor (optional)	LLM	Event log only	Answer why questions with citations	Anything else



agent flow · known path down the middle, learning loop on the right

Known breaks run straight down the middle. A break with no matching playbook detours through teach mode, and the approved playbook joins the library for next time.

Rules enforced in code (each has a test)

- Each agent receives only its own tools; an out-of-scope call is denied and logged.
- A proposal's prepared_by can never equal its approved_by.
- An approval stores a SHA-256 hash of the proposal; the executor refuses if the content changed.

- Only the executor holds ledger and bank credentials.
- Every action writes an event: actor, time, case, inputs, evidence references, short reason.

Playbooks and the learning loop

Each playbook (workflow) owns one break key, meaning which sources disagree and on what field, so choosing it is a lookup with no inference. Inside, it branches on facts from the other sources. Learning adds a playbook or a branch that a controller approves, never model weights, so every learned behavior stays readable and revocable.

Playbook schema (one per break key)

```
id: wf-ledger-custodian-qty
version: 3
status: active            # draft | active | flagged | retired
key: {sources: [ledger, custodian], field: qty}   # picked by lookup, no
inference
context: [executions_qty, corporate_actions, client_notified]
branches:                 # checked in order; exactly one may match
  - id: split
    when: split_on_symbol and custodian_qty == ledger_qty * ratio
    trust: custodian
    actions: [split_adjust]
    provenance: {taught_by: Dana Ortiz, from_case: CASE-0217, approved_by:
Sam Lee}
  - id: partial-fill
    when: executions_qty == custodian_qty
    trust: executions
    actions: [reverse_unfilled, notify_client_service]   # amounts from
calculators
    provenance: {taught_by: Dana Ortiz, from_case: CASE-0142, approved_by:
Sam Lee}
  - id: broker-report
    when: executions_qty == ledger_qty
    trust: ledger
    actions: [request_info_custodian, hold]
  - else: needs_human
modifiers:
  - when: client_already_notified
    skip: [notify_client_service]
```

```
approval: {auto_under_usd: 500, otherwise: controller}
verify: rerun_recon(affected_keys) == 0 breaks
stats: {split: {used: 1, succeeded: 1}, partial-fill: {used: 11, succeeded:
11}}
```

Context step. Before branching, deterministic code pulls the facts listed under context from the other sources into the case. Branch conditions read only those facts, so the same case always takes the same branch. Rules that apply to every playbook are global policies, not branches: a fix for a closed period posts in the open period with a reference, materiality, and the $500 approval limit.

Action library (a closed set; playbooks only compose these): journal_entry (templates: reverse, adjust, dividend, split_adjust), source_correction_request, transfer, request_info, notify, hold, escalate. A new action type needs code, not learning.

The loop

1. **Induction.** Load about 30 resolved Q2 cases plus an SOP document. The Learner groups them by break key and drafts one playbook per key, with a branch per distinct fix (B1 to B4). The controller approves. The demo starts with this done, history visible.
2. **Known break.** Look up the playbook by break key, run the context step, and take the one branch whose condition holds. The LLM confirms the branch fits the evidence and explains it; it never sets amounts. Preparer builds proposals, approval follows policy, executor applies, verify reruns recon on the affected keys.
3. **New break.** No playbook for the key, or the case lands on else, means Needs human. In teach mode the human picks the trusted source, chooses actions from the library, and names the fact that makes this case different. The fix runs once, through approval. The Learner drafts a new branch (or a new playbook for a new key) with that fact as its condition, plus a test built from the case. The controller approves.
4. **Conflicts.** If two branches match, their conditions overlap: escalate, and the human tightens one. If a human resolves a matched case differently, the Learner asks whether it is a different cause (new branch) or the same cause in a different situation (new modifier).
5. **Improve.** A branch whose verify fails, or that a human overrides, is flagged for revision. Usage and success counts show per branch.

Case statuses: New, Investigating, Waiting on someone, Awaiting approval, Completed, Auto-cleared, Needs human. Your in processing maps to the three middle states; handled folds into Auto-cleared. A Completed case must cite the playbook id, version and branch, the proposals executed and the verify result.

Time-series automation

A simulated clock replays the quarter one business day at a time, so breaks age, periods close and lessons from August change what happens in October.

- **Daily loop:** ingest that day's source data, run recon, let agents work open cases, run end-of-day position and cash recon, then aging checks. Command: make replay FROM=2026-07-01 TO=2026-10-23 SPEED=fast.
- **Three recon levels:** transaction (orders, executions, ledger, custodian settlements), position (cumulative trades plus corporate actions vs custodian position, daily), cash (ledger vs custodian vs bank).
- **Aging:** Waiting on someone gets a follow-up after 1 business day and goes to the controller after 3.
- **Period close:** each month locks on its 3rd business day. A break found later is corrected in the open period with a reference to the original entry; closed periods are never edited.
- **Memory across time:** cases, playbooks and their history persist across the replay, so the October split uses the August playbook.
- **Demo speed:** the full replay should run in under 2 minutes; pause on any day from the header controls.

Dashboard

Eight P0 pages carry the demo; build P1 only if time allows. Replay controls (play, pause, speed, simulated date) sit in the header on every page.

Page	Shows	Priority
Overview	Open cases by status, dollars at risk, auto-resolution rate, median time to resolve, breaks per day across the quarter, aging buckets	P0
Breaks	Filterable table: id, type, account, symbol, amount, sources, detected date, age, status, agent, playbook	P0

Page	Shows	Priority
Break detail	Source values side by side with differences highlighted; agent timeline with reasons and evidence links; proposals, approvals, messages; completion note citing the playbook	P0
Approvals	Controller queue: approve or reject, with the evidence beside it	P0
Teach mode	Walk a novel case: pick trusted source, choose actions, write a note, review the draft playbook	P0
Playbooks	Library with version history, provenance, usage stats; approve, flag, retire	P0
Proof	Detection precision and recall per type vs the answer key, resolution correctness, ledger invariants (all green)	P0
Inject break	Judge picks a type, account and symbol; it lands in the live replay	P0
Sources	Each source's tables, raw	P1
Inbox	Messages with personas (also shown inside break detail)	P1
Audit	Searchable event log plus an ask-why box answered with citations	P1

Requirements

28 requirements, numbered so every test and build step can point at them.

World and data

- R1. Seeded generator: the same seed produces byte-identical sources and answer key.
- R2. Five systems (OMS, executions, ledger, custodian, bank) plus reference data, fee schedule and directory, each behind its own API with its own key.
- R3. About 80 business days (Jul 1 to Oct 23, 2026), 200 accounts, 40 symbols from SEC EDGAR, roughly 24k orders.
- R4. The injector plants the taxonomy above and writes the answer key.

- R5. A clean mode with zero injected breaks.

Ledger and accuracy

- R6. All money and quantities are Decimal in Python and NUMERIC in the database; floats are banned.
- R7. Every journal entry balances per asset; the trial balance is zero at every end of day.
- R8. Posted entries are immutable; corrections are new entries that reference the original.
- R9. Closed periods reject postings; late fixes post to the open period.
- R10. Every posting and transfer has an idempotency key; a replay never double-posts.
- R11. Proposal amounts come only from deterministic calculators.

Detection

- R12. The deterministic matcher covers transaction, position and cash recon.
- R13. Harmless differences (T+1 timing, rounding of $0.01 or less) auto-clear with a stated reason.
- R14. Rerunning recon on the same data creates no duplicate breaks.

Agents and segregation

- R15. Agents as defined above, each limited to its own tools; denied calls are logged.
- R16. Preparer never equals approver; approval is bound to the proposal hash; only the executor writes.
- R17. The approver agent handles only proposals of $500 or less from active playbooks; everything else goes to the controller.
- R18. Outreach routes by directory role, follows up after 1 business day, escalates after 3.

Learning

- R19. Playbooks follow the schema: one per break key, ordered branches and modifiers, versions, and per-branch provenance and stats. Branch conditions read only facts gathered by the deterministic context step.
- R20. Induction drafts initial playbooks from historical cases and the SOP; the controller approves them.
- R21. No playbook for the break key, or a case on the else branch, means Needs human; a teach session produces a draft branch or playbook, active only after approval.
- R22. A failed verify or a human override flags the playbook.

Time, audit, UI, generality

- R23. Replay runs day by day; aging and period close behave as specified.
- R24. An append-only event log records every state change with actor, time, inputs, evidence and reason.
- R25. All P0 dashboard pages work.
- R26. A judge can inject a break from the UI and watch it handled live.
- R27. The Proof page reports detection and resolution accuracy against the answer key.
- R28. A new domain loads through connector config, schema mapping and induction, with no core code changes.

Tests (write before building)

37 tests. Deterministic parts must pass every time. LLM-dependent tests (T26, T27, T32) run 3 times at temperature 0 and must pass 3 of 3; the verify step and approvals make sure a wrong judgment can never post a wrong amount.

ID	Test	Passes when	Covers
T1	Determinism	Two runs with seed 42 produce identical file hashes	R1
T2	True world consistency	Positions rebuilt from trades plus corporate actions equal true positions every day	R3
T3	Answer key completeness	Every injected break has a key row with records and the expected fix	R4
T4	Clean world	Recon on clean mode finds 0 breaks	R5, R12
T5	No floats	Scan of money and quantity fields finds no float types	R6
T6	Balanced entries	Every entry has debits equal to credits per asset; an unbalanced entry is rejected	R7
T7	Trial balance	Sums to zero after every replay day	R7
T8	Immutability	Updating or deleting a posted entry is rejected	R8
T9	Period lock	Posting into closed September is rejected; the fix lands in October with a reference	R9

ID	Test	Passes when	Covers
T10	Idempotency	Executing the same approved proposal twice yields one posting and one transfer	R10
T11	Calculator-only amounts	A proposal amount not produced by a calculator is rejected	R11
T12	Detect each type	One test per B1 to B4, N1, N2: detected with the right type and keys	R12
T13	Auto-clear	T+1 and rounding cases auto-clear with a reason and never escalate	R13
T14	Full detection accuracy	Full world vs answer key: 100% of planted breaks found, 0 unplanted breaks	R12, R13
T15	Rerun idempotent	Running recon twice creates no duplicate breaks	R14
T16	Tool scoping	Each agent calling a tool outside its scope is denied and logged	R15
T17	Self-approval	A preparer approving its own proposal is rejected	R16
T18	Tampered proposal	Editing a proposal after approval makes the executor refuse it	R16
T19	Write isolation	Any non-executor attempt to post or transfer fails for lack of credentials	R16
T20	Approval limit	A $501 proposal or a new playbook sent to the approver agent goes to the controller	R17
T21	Routing	Each break type's outreach reaches the role named in its playbook	R18
T22	Follow-up and escalation	With no reply, follow-up fires at day 1 and escalation at day 3	R18
T23	Playbook schema	An invalid playbook is rejected with field errors	R19

ID	Test	Passes when	Covers
T24	Playbook match	Each break key routes to its playbook and each known case hits exactly one branch; novel cases hit else; overlapping branches escalate	R19, R21
T25	Drafts cannot run	A draft playbook is never used; the case goes to Needs human	R21
T26	Induction	30 historical cases plus the SOP produce drafts for B1 to B4 with the expected actions	R20
T27	Learning loop end to end	First split hits else, teaching adds a split branch, the controller approves, then the second split completes automatically, citing the branch and teacher, with correct amounts	R21
T28	Revision flag	A failed verify or a controller override flags the playbook	R22
T29	Resolution correctness	After replay, resolved items' ledger state equals the true world; proposal amounts equal the answer key exactly	R11, R27
T30	Replay	80 days finish without errors; aging and escalations fire as specified	R23
T31	Audit completeness	Every state change has an event with actor, time, evidence and reason	R24
T32	Ask why	A why question about a completed case is answered citing its events and playbook	R24
T33	Live injection	A B2 injected from the UI is detected and handled within one replay day	R26
T34	Proof page	Shown metrics equal the eval script's output	R27
T35	Generality	BenchRec loads by config; recon and induction run; no core code diff	R28
T36	Demo script	make demo ends with every case in its expected state	All

ID	Test	Passes when	Covers
T37	UI smoke	Every P0 page loads with data and no console errors	R25

Build plan for Claude Code

18 steps, about 32 hours of work, so split it across the team or take the scope cuts below. Give Claude Code one step at a time with its requirements and tests; have it write the tests first, code until they pass, then commit.

Rules for CLAUDE.md: Decimal only, no floats. Tests before code. Only the executor writes to the ledger or bank. Never edit a closed period. Every state change emits an event. LLM output never sets an amount.

Tracks (if you have teammates): A = steps 1 to 5, B = 6, 7, 9, C = 8 and 10 to 13, D = 14 and 15 (starts after step 1, using fixtures).

1. **Step 0 — Setup** (0.5 h)
   - Build: repo layout, docker-compose (Postgres, Formance), Makefile, CLAUDE.md. Pick the agent framework.
   - Done when: make up runs and an empty test suite passes.
2. **Step 1 — Contracts** (1 h)
   - Build: Pydantic models for every source plus Break, Case, Proposal, Playbook, Event, AnswerKey; migrations; one OpenAPI spec per source.
   - Done when: T5, T23.
3. **Step 2 — True world generator** (2.5 h)
   - Build: accounts, EDGAR symbols, prices, orders, fills including partials, fees per schedule with one mid-quarter tier change, two splits, dividends, ACH deposits and withdrawals; derive each source's view.
   - Done when: T1, T2.
4. **Step 3 — Break injector and answer key** (1.5 h)
   - Build: plant the taxonomy at the listed counts; clean mode.
   - Done when: T3 (T4 after step 6).
5. **Step 4 — Source services** (1.5 h)

- Build: FastAPI mocks for OMS, executions, custodian and reference data, served by simulated date, each with its own key; Increase sandbox connector for ACH in, out and returns.
- Done when: each service returns a given day's data and rejects other services' keys.

6. **Step 5 — Ledger** (1.5 h)
   - Build: Formance setup, chart of accounts (client cash, client positions by symbol, firm omnibus, fee revenue, client receivables), posting service with idempotency keys, period lock. Switch to the Postgres fallback if Formance takes more than an hour.
   - Done when: T6 to T10.
7. **Step 6 — Matcher** (2 h)
   - Build: transaction, position and cash recon; tolerance and timing rules; breaks table.
   - Done when: T4, T12 to T15.
8. **Step 7 — Replay engine** (1 h)
   - Build: simulated clock, daily loop, aging, month-end lock.
   - Done when: T9 and T30 without agents.
9. **Step 8 — Agent runtime and permissions** (2 h)
   - Build: llm.py wrapper, tool registry with per-agent scopes, event logger, deterministic calculators per action template.
   - Done when: T11, T16, T19, T31.
10. **Step 9 — Proposals, approvals, executor** (1.5 h)
    - Build: proposal hashing, approver policy, controller queue API, executor.
    - Done when: T10, T17, T18, T20.
11. **Step 10 — Playbooks and induction** (2.5 h)
    - Build: schema, action library, 30 historical Q2 cases plus SOP.md, Learner, playbook matching.
    - Done when: T23 to T26.
12. **Step 11 — Investigator, Preparer, verify** (2 h)
    - Build: end-to-end flow for known breaks.
    - Done when: T14, and T29 for B1 to B4.
13. **Step 12 — Outreach and personas** (1.5 h)
    - Build: directory, inbox, persona agents that answer from the true world, follow-ups.
    - Done when: T21, T22.

14. **Step 13 — Teach mode and learning loop** (2 h)
    - Build: Needs human state, teach API, Learner from a session, approval, revision flag.
    - Done when: T27, T28.
15. **Step 14 — Dashboard** (4 h)
    - Build: the P0 pages.
    - Done when: T37.
16. **Step 15 — Proof, inject, ask why** (1.5 h)
    - Build: eval script and Proof page, inject-break endpoint and form, Auditor.
    - Done when: T32 to T34.
17. **Step 16 — Generality (stretch)** (1.5 h)
    - Build: BenchRec connector config and induction run.
    - Done when: T35.
18. **Step 17 — Demo hardening** (2 h)
    - Build: make demo script, video that opens with the split moment, README with the real vs simulated table.
    - Done when: T36.

Checkpoints: Saturday noon: T1 to T15 green. Saturday 8 pm: known breaks resolve end to end on the dashboard. Saturday 11 pm: Devpost draft submitted. Sunday 3 am: learning loop works. Sunday 7 am: code freeze, record the video. Sunday 9 am: final submission.

Scope cuts and submission

If you fall behind, cut from the top of this list; never cut the answer key, the segregation checks or the learning loop.

1. Generality demo (BenchRec)
2. Auditor and ask-why
3. Sources, Inbox and Audit pages (show messages inside break detail instead)
4. Formance, replaced by the Postgres ledger
5. N2, the ACH return (keep the split as the moment)
6. Increase, replaced by the mock bank

Real vs simulated (goes in the README and the pitch)

Part	Status
Symbols and CIKs	Real, from SEC EDGAR
Prices	Real if fetched, otherwise simulated (labeled)
Ledger engine	Real software (Formance), simulated entries
Bank	Real API (Increase sandbox), sandbox money
Orders, executions, custodian	Simulated by the seeded generator
People	LLM personas answering from the true world
Breaks	Planted, with an answer key

Submission checklist

- [ ] Devpost draft submitted Saturday night
- [ ] Demo video opens with the split moment
- [ ] One line on seats: trade reconciliation analyst and settlements ops analyst at a retail brokerage
- [ ] Proof: answer-key metrics plus the real vs simulated table
- [ ] README: setup, architecture, where each agent acts
- [ ] Rehearse a judge-injected break
- [ ] Whole team at the table from 10:30 am to 12:30 pm Sunday